MEMORIA TÉCNICA

Gestión de Incidencias con Power Platform e IA

Fase 3: operación, seguridad y gobierno — y hoja de ruta a producción

Azure Key Vault · Entra ID · RBAC · Log Analytics · Cost Management · Dataverse · Power Automate · Teams (tarjetas adaptables)

José Rafael Vázquez Silva
Proyecto personal de portafolio
Octubre de 2026

AudioVital es una marca ficticia. Todos los clientes, incidencias y documentos son inventados para este proyecto y no corresponden a ninguna empresa real.

Índice

1. Resumen ejecutivo.....3
2. Punto de partida y objetivo de la Fase 3.....4
3. Modelo de uso: quién usa y quién gestiona la solución.....5
4. Arquitectura final (Fases 1 a 3).....6
5. Seguridad: Key Vault, entidad de servicio y RBAC.....7
6. Observabilidad y control de costes.....9
7. Incidente: bucle de autodisparo del Flujo 2.....11
8. Ciclo de vida de la incidencia en Dataverse.....14
9. Flujo 3 — Cierre con resolución obligatoria.....15
10. Flujo 4 — Asignación «Me encargo» con tarjeta adaptable.....17
11. Problemas resueltos en la Fase 3.....19
12. Lo que no se construyó.....20
13. Límites conocidos (consolidados).....21
14. Hoja de ruta para llevarlo a producción.....22
15. Evidencias.....25

1. Resumen ejecutivo

Este documento continúa la *Memoria técnica de las Fases 1 y 2* del proyecto **Gestión de Incidencias con Power Platform e IA** (marca ficticia **AudioVital**). Al cerrar la Fase 2, la solución clasificaba incidencias con Azure OpenAI y respondía en Teams fundamentándose en documentación corporativa (RAG con Azure AI Search). Funcionaba, pero tenía problemas de **operación y gobierno**: una clave de API pegada en un flujo, ninguna visibilidad sobre consumo, costes ni errores, y una incidencia que se creaba y clasificaba, pero no tenía dueño ni forma de cerrarse.

La **Fase 3** resuelve esos tres frentes:

Frente	Antes (Fase 2)	Después (Fase 3)
Secretos	Clave de Azure AI Search escrita en el paso HTTP	Clave de **consulta** (solo lectura) guardada en **Azure Key Vault** y leída por una **entidad de servicio** con RBAC de mínimo privilegio
Observabilidad	Ninguna: la cuota semántica agotada se descubrió por casualidad	**Métricas** de tokens, **presupuesto con alertas** y **diagnósticos** de Azure OpenAI en **Log Analytics**, consultables con KQL
Coste y fiabilidad del asistente	Respuestas de hasta 5 min, duplicadas, errores 429	Bucle de autodisparo cortado y carga del prompt reducida un **80 %**: una respuesta en 8–15 s
Ciclo de vida	Alta → clasificación → aviso. Sin dueño ni cierre	Autor y propietario reales, **asignación desde Teams** («Me encargo»), **cierre con resolución obligatoria** y aviso al autor

Se construyeron **dos flujos nuevos** (cierre y asignación) y se modificaron los dos existentes. Todo está probado en el entorno real con evidencias (capturas 17 a 61). La memoria termina con lo que **no** se llegó a construir y con una **hoja de ruta concreta para llevar la solución a producción** en una organización real.

> *Proyecto personal de portafolio. Todos los datos, clientes, documentos y la marca AudioVital son ficticios. El entorno es real: un inquilino de formación con crédito de Azure temporal y una prueba de Microsoft Teams.*

2. Punto de partida y objetivo de la Fase 3

La pregunta de la Fase 3 fue: **¿qué le falta a esta solución para que un equipo de IT la pudiera operar con tranquilidad?** Del análisis salieron dos bloques, ordenados por la fecha de caducidad del crédito de Azure (~21/10/2026):

Bloque A — Azure (seguridad, observabilidad y coste)

- Sacar la clave del flujo a **Azure Key Vault** y leerla con una identidad propia, no con la cuenta del autor.
- **RBAC por recurso** con el principio de mínimo privilegio.
- **Diagnósticos** de Azure OpenAI hacia Log Analytics, **métricas de tokens** y **presupuesto con alertas**.

Bloque B — Power Platform (ciclo de vida de la incidencia)

- Sustituir el «agente asignado» escrito a mano por la **identidad real** de quien registra y de quien gestiona.
- Nuevos estados (*En curso*, *Pendiente de revisión*) y columna **Resolución**.
- **Cierre controlado**: no se puede cerrar sin resolución y el autor recibe el aviso.
- **Asignación** de las escaladas desde Teams con una tarjeta adaptable.

Durante el Bloque A apareció un incidente real (el bucle de autodisparo del Flujo 2, sección 7), que se trató como prioridad antes de seguir.

3. Modelo de uso: quién usa y quién gestiona la solución

La solución es una **herramienta interna**. El cliente final no entra en Power Apps ni en Teams: habla con la tienda o con atención telefónica. Así lo exige además el protocolo de atención de la base documental, que prohíbe exponer campos internos como la prioridad o el escalado.

Rol	Qué hace en el sistema	Herramienta
Agente (tienda o teléfono)	Registra la incidencia con sus palabras; consulta al asistente dudas sobre garantías, citas o reclamaciones; cierra la incidencia con su resolución	Power Apps · Teams
Supervisor	Recibe las escaladas (prioridad Alta) y se las asigna con «Me encargo»; revisa clasificaciones fallidas	Teams (canal) · Power Apps
Responsable de zona / dirección	Sigue volumen, tipos, prioridades y estados	Power BI

Tres responsabilidades separadas

- **Atención al cliente es dueña del contenido:** mantiene los PDFs de políticas y los criterios de prioridad. Cambiar una política no exige tocar ningún flujo, solo volver a indexar.
- **IT / administración de Power Platform es dueña de la plataforma:** licencias, entornos, permisos, Key Vault, presupuesto y vigilancia de los flujos.
- **Calidad:** repite la evaluación de la clasificación con incidencias nuevas cada vez que cambia el prompt o el modelo.

Principio de diseño: el aviso avisa, la lista de trabajo vive en Dataverse

Un mensaje de Teams se pierde entre otros. Por eso Teams solo **notifica** y permite **actuar** (botón «Me encargo»), pero el estado real de cada incidencia, su autor, su propietario y su resolución viven siempre en **Dataverse**, que es la fuente única de verdad.

4. Arquitectura final (Fases 1 a 3)

```
Power Apps ──► Dataverse (fuente única de verdad)
                  │
                  ├─► Flujo 1  Alta ──► Azure OpenAI (clasificación)
                  │       ├─► escribe Tipo, Prioridad, Estado, Fecha_Alta
                  │       └─► Teams: aviso de escalado / de fallo
                  ├─► Flujo 3  Cerrada ──► exige Resolución, Fecha_Cierre,
                  │                        aviso al autor por Teams
                  ├─► Flujo 4  Escalada ──► tarjeta «Me encargo»
                  │                        ──► Propietario + En curso
                  └─► Power BI

Teams ──► Flujo 2  Asistente
            ├─► Dataverse (50 últimas incidencias)
            ├─► Key Vault (entidad de servicio) ──► clave de consulta
            ├─► Azure AI Search (híbrida) ◄── Blob Storage + embeddings
            └─► Azure OpenAI ──► respuesta citando la fuente

Azure OpenAI ──► Diagnósticos ──► Log Analytics (KQL)
Cost Management ──► presupuesto y alertas
```

Flujo	Desencadenador	Qué hace	Fase
1 · Alta de incidencia	Fila agregada en Incidencias	Clasifica con IA (temperatura 0, JSON), escribe Tipo, Prioridad, Escalada, Estado y Fecha_Alta; avisa por Teams si es Alta o si falla	1 · 2 · 3
2 · Consulta (asistente)	Palabra clave en el canal de Teams	Corta su propio eco, lee 50 incidencias, obtiene la clave de Key Vault, busca en AI Search y responde con Azure OpenAI citando la fuente	1 · 2 · 3
3 · Cierre con aviso	Estado cambia a Cerrada	Si falta la resolución, devuelve a En curso; si no, fecha el cierre y avisa al autor por Teams	3
4 · Escalado «Me encargo»	Estado cambia a Escalada	Publica una tarjeta con botón en el canal; quien pulsa pasa a ser propietario y la incidencia queda En curso	3

Recursos de Azure (grupo rg-incidencias-audiovital, East US)

Recurso	Nombre	Función
Azure OpenAI	`openai-incidencias-rafael`	`gpt-4.1-mini` (100k TPM, 100 RPM) y `text-embedding-3-small`
Azure AI Search (Gratis)	`audiovital-search`	Índice `audiovital-rag` (11 fragmentos de 6 PDFs)
Storage Account (LRS)	`audiovitalstorage`	Contenedor privado `base-documental`
Key Vault (Estándar, RBAC)	`kv-audiovital-rvs`	Secreto `search-query-key`
Log Analytics	`log-audiovital`	Destino del diagnóstico `diag-openai`
Entra ID · registro de aplicación	`app-audiovital-flujos`	Entidad de servicio con la que el Flujo 2 lee Key Vault
Cost Management	`presupuesto-audiovital`	25 €/mes con alertas

5. Seguridad: Key Vault, entidad de servicio y RBAC

5.1 Problema

En la Fase 2 el paso HTTP del Flujo 2 llevaba la clave de Azure AI Search escrita en la cabecera `api-key`. Cualquiera con permiso de edición sobre el flujo, o acceso a su historial, podía leerla. Rotarla obligaba a editar el flujo.

5.2 Qué se construyó

1. **Registro de aplicación en Entra ID** (`app-audiovital-flujos`) con un secreto de cliente (`kv-lectura-flujos`, caduca el 13/1/2027). Es la identidad con la que el flujo lee el almacén: no depende de la cuenta personal del autor.
2. **Key Vault** `kv-audiovital-rvs`: plan Estándar, **modelo de permisos RBAC** (no directivas de acceso), retención de borrado temporal de 7 días.
3. **Asignaciones de rol con ámbito «este recurso»:** la cuenta del autor tiene *Agente de secretos de Key Vault* (crea y gestiona secretos); la aplicación tiene **solo** *Usuario de secretos de Key Vault* (lee secretos, nada más).
4. **Secreto `search-query-key`** con la **clave de consulta** de Azure AI Search, no la de administración: aunque se filtrara, solo permite buscar, no modificar ni borrar el índice.
5. **Flujo 2:** nueva acción **Obtener secreto** (conector Azure Key Vault, conexión `kv-audiovital-entidad-servicio` con autenticación de entidad de servicio) antes del paso HTTP. La cabecera `api-key` pasa a ser `body/value` del secreto.
6. **Entradas y salidas seguras** activadas en *Obtener secreto* y en *HTTP*: el historial de ejecuciones muestra «Contenido no mostrado debido a la configuración de seguridad».

5.3 Resultado

Prueba de punta a punta en Teams: *Obtener secreto* 0,7–0,8 s, *HTTP* 0,4–0,7 s y respuesta correcta del asistente. La clave ya no aparece en el flujo, en su definición ni en su historial.

5.4 Decisiones y compromisos

- **Clave de consulta en lugar de la de administración:** mínimo privilegio también en el secreto, no solo en quién lo lee.
- **Las salidas seguras se propagan:** como el resultado del paso HTTP es seguro, las entradas del paso de Azure OpenAI también salen ocultas. Ganamos seguridad, pero perdemos la visibilidad del prompt al depurar. Es un compromiso consciente.
- **No es una identidad administrada pura.** Los flujos de nube de Power Automate no tienen identidad administrada para este conector; la entidad de servicio usa un secreto de cliente, guardado cifrado en la conexión. Rotarlo es una tarea pendiente de automatizar.
- **Acceso público al Key Vault:** Power Automate no sale desde una IP fija, así que el almacén admite conexiones desde todas las redes, protegido por Entra ID y RBAC. En producción se cerraría (sección 14).

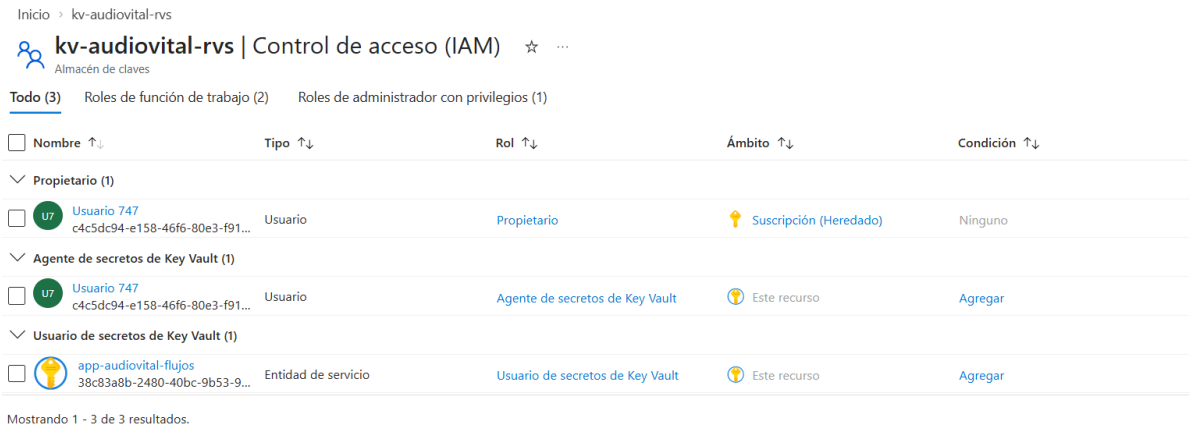


Figura 1. *Asignaciones de rol en el Key Vault: la aplicación solo puede leer secretos.*

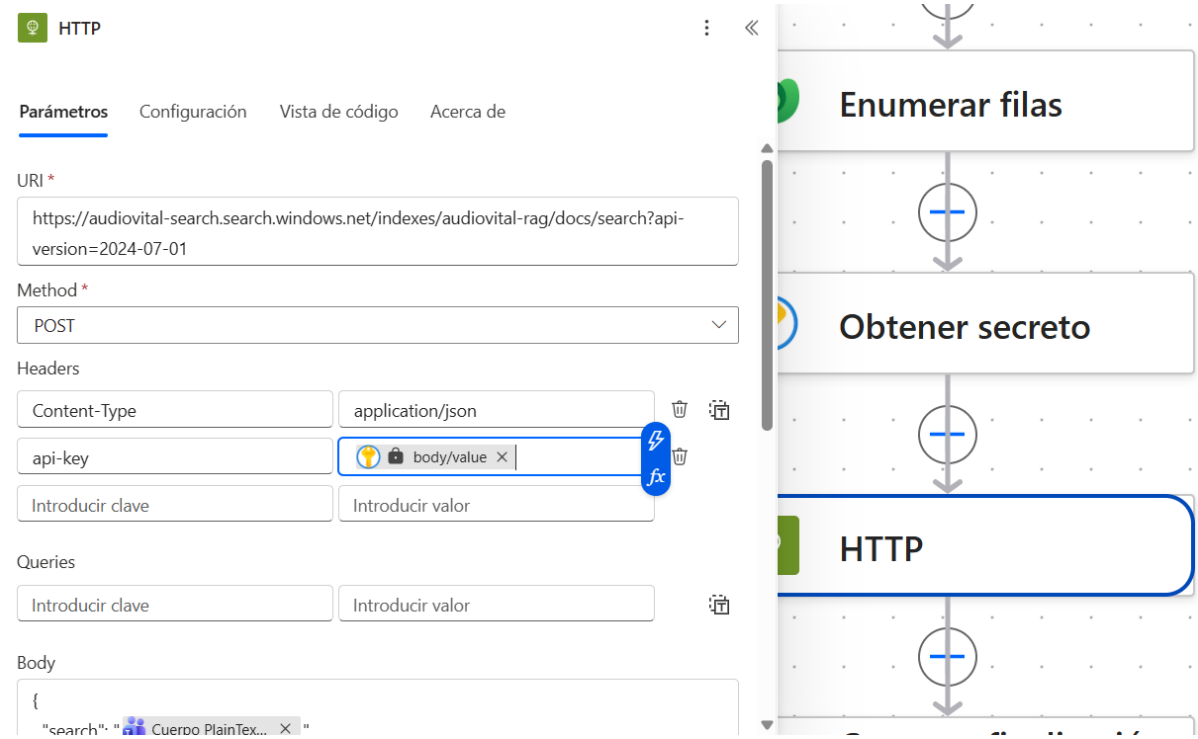


Figura 2. *Flujo 2: la cabecera api-key toma el valor del secreto leído de Key Vault.*

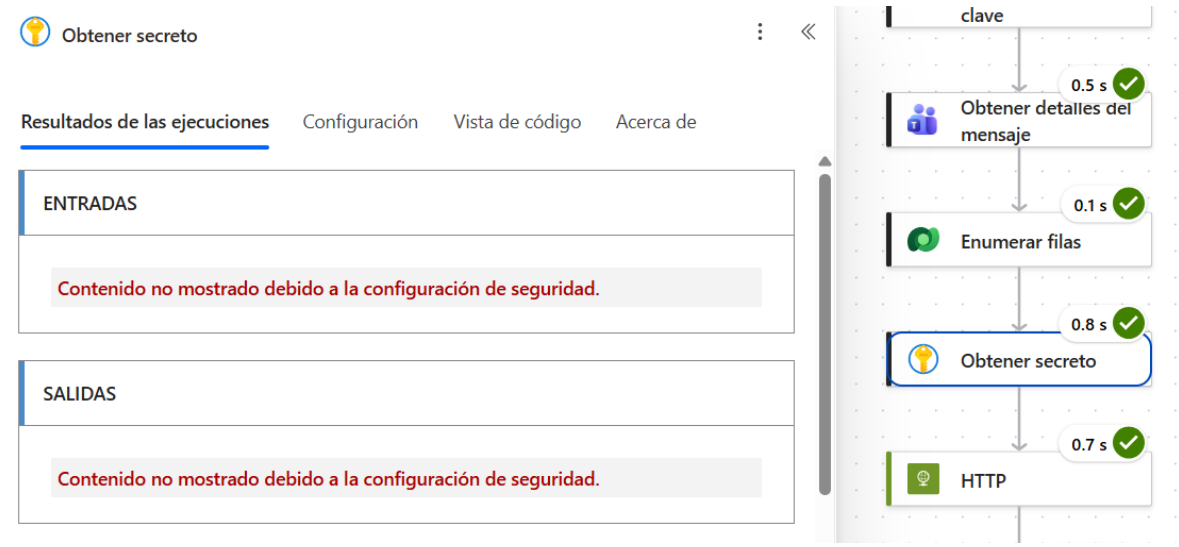


Figura 3. *Entradas y salidas seguras: el secreto no aparece en el historial.*

6. Observabilidad y control de costes

La Fase 2 dejó una lección: la cuota semántica gratuita se agotó y el asistente dejó de funcionar **sin que nadie lo supiera**. La Fase 3 añade tres capas para ver qué pasa y cuánto cuesta.

6.1 Métricas del recurso de Azure OpenAI

La métrica *Processed Inference Tokens* mostró el consumo real. Fue la que reveló el incidente de la sección 7: una meseta de ~150.000 tokens cada 5 minutos, **36,53 millones de tokens en 24 horas**, y la caída al corregirlo.

6.2 Coste por recurso y presupuesto

- **Análisis de costes** por recurso: octubre pasó de 9,62 € a 19,55 €, casi todo en Azure OpenAI. Key Vault, Storage y AI Search: céntimos o cero.
- **Presupuesto `presupuesto-audiovital`**: 25 €/mes, alerta por correo al **80 % del gasto real** y al **100 % del previsto**. Un bucle como el de la sección 7 habría disparado la alerta en lugar de descubrirse revisando métricas.

6.3 Diagnósticos en Log Analytics

Configuración de diagnóstico `diag-openai` en el recurso de Azure OpenAI, enviando las categorías **Audit**, **Request and Response** y **Azure OpenAI Request Usage**, además de **AllMetrics**, al área de trabajo `log-audiovital`. Consulta de verificación:

```
AzureDiagnostics
| where TimeGenerated > ago(1h)
| project TimeGenerated, OperationName, ResultSignature, DurationMs
| order by TimeGenerated desc
```

Resultado: una pregunta en Teams aparece como `post` (200) → `Embeddings_Create` (200, 175 ms) → `chatcompletions_create` (200, 1.079 ms). Es decir, se ve **cada llamada del RAG**: la vectorización de la consulta y la generación de la respuesta, con su código y su latencia. Los 429 de la sección 7 habrían aparecido aquí con `ResultSignature = 429`.



Figura 4. *Log Analytics: cada pregunta genera una llamada de embeddings y otra de chat, ambas con código 200.*

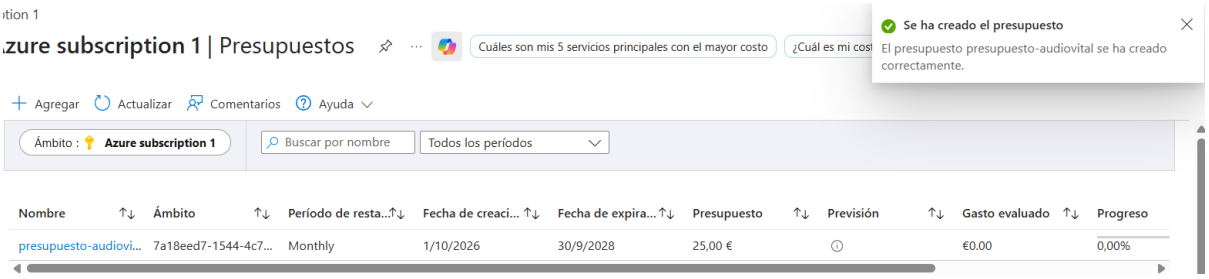


Figura 5. *Presupuesto mensual de 25 € con alertas por correo.*

7. Incidente: bucle de autodisparo del Flujo 2

7.1 Síntoma

Desde la tarde del 15/10 el asistente tardaba hasta **5 minutos** en contestar y a veces respondía **dos o tres veces**. El historial mostraba ejecuciones cada ~5 minutos, también de noche, sin que nadie escribiera en el canal.

7.2 Diagnóstico, paso a paso

1. El **historial de reintentos** del paso de Azure OpenAI mostraba un primer intento con **429** (demasiadas peticiones) y un segundo correcto tras la espera automática de Power Automate: de ahí los 5 minutos.
2. Se comprobó el **límite del despliegue**: 100.000 tokens y 100 peticiones por minuto. No era bajo, así que el límite no era la causa sino el síntoma.
3. El desencadenador escucha la palabra clave **«Incidencias»**… y las respuestas del asistente también la contienen. **El asistente se disparaba a sí mismo.**
4. Confirmado en la salida de *Obtener detalles del mensaje*: `from.user = null` y `from.application.displayName = "Workflows"`, con `applicationIdentityType = "bot"`.
5. Agravante: *Enumerar filas* sin filtro enviaba **247.625 bytes** (~60–70k tokens) en cada llamada. Dos llamadas en el mismo minuto superaban los 100k TPM.

7.3 Corrección

1. Flujo **desactivado** de inmediato para cortar el gasto.
2. **Condición antibucle** justo después de *Obtener detalles del mensaje*: `empty(body('Obtener_detalles_del_mensaje')?['from']?['user'])` es igual a `true` → **Finalizar (correcto)**. Si el mensaje no lo escribe una persona, el flujo termina sin llamar a la IA.
3. **Enumerar filas filtrado:** solo 12 columnas necesarias, orden `createdon desc` y **50 filas** como máximo.

7.4 Resultado

Medida	Antes	Después
Tiempo de respuesta	~5 min (con 429)	**8–15 s**
Respuestas por pregunta	2–3	**1**
Autodisparo	Llamaba a la IA cada vez	Termina en < 1 s sin llamar a la IA
Carga de Enumerar filas	247.625 bytes	**48.387 bytes (−80 %)**
Tokens	36,53 M en 24 h	Caída inmediata a niveles normales

7.5 Lección

Un 429 no siempre significa «sube la cuota». Aquí la causa era de **diseño** (un desencadenador que oye sus propias respuestas) y de **tamaño del prompt** (enviar la tabla entera). Corregir la causa fue gratis; subir la cuota habría ocultado el problema y multiplicado el coste. Además, sin métricas ni presupuesto el bucle habría seguido consumiendo crédito en silencio: por eso las piezas de la sección 6 no son decorativas.



Figura 6. *El mensaje que dispara el flujo lo envió el bot Workflows: from.user es null.*



Figura 7. *Tokens procesados: meseta cada 5 minutos durante el bucle y caída tras la corrección.*

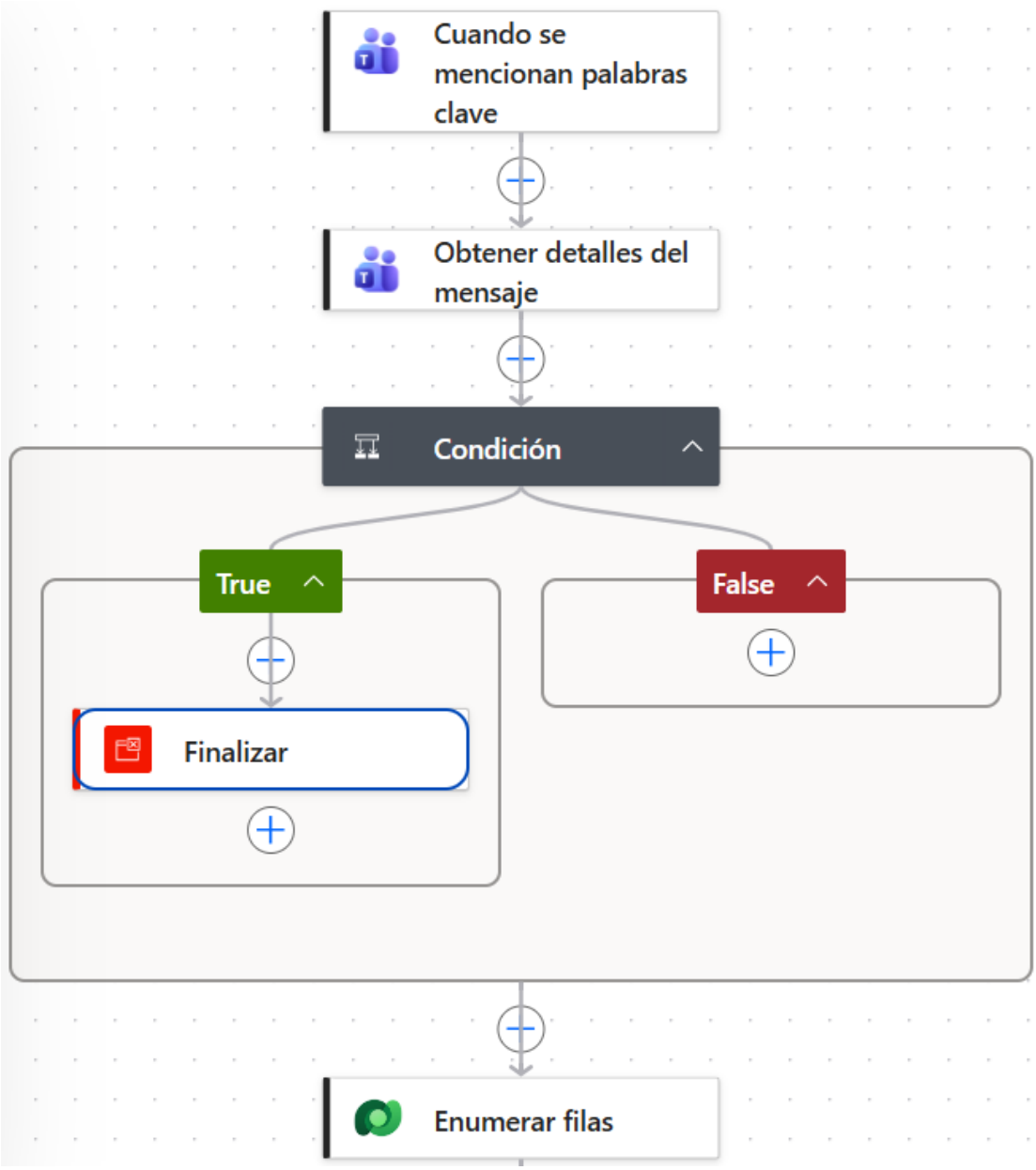


Figura 8. *Condición antibucle: si no escribe una persona, el flujo termina.*

8. Ciclo de vida de la incidencia en Dataverse

8.1 De «agente asignado» a identidad real

La Fase 1 usaba una columna de opción *Agente asignado* (Agente 1/2/3) que se rellenaba a mano: no identificaba a nadie y no servía para permisos. En la Fase 3 se sustituye por las columnas de sistema de Dataverse:

- **Creado por (Autor):** quien registra la incidencia. Automático e inmutable. Se muestra en el formulario como campo de solo lectura.
- **Propietario:** quien la gestiona. Es la columna que usan los roles de seguridad de Dataverse y la que cambia el botón «Me encargo».

La columna antigua se quitó del formulario, pero se mantiene en la tabla por las incidencias históricas.

8.2 Estados y columnas nuevas

Estado	Valor	Quién lo escribe
Abierta	610150000	Valor por defecto de la columna; Flujo 1 si la prioridad no es Alta
En curso	610150001	Flujo 4 (al asignarse) o Flujo 3 (si se intenta cerrar sin resolución)
Escalada	610150002	Flujo 1 cuando la prioridad es Alta
Cerrada	610150003	El agente, desde la app (solo se mantiene si hay resolución)
Pendiente de revisión	610150005	Creado para la rama de error del Flujo 1 (conexión pendiente, sección 12)

- **Resolución:** texto de varias líneas, en el formulario junto a Fecha_Cierre.
- **Fecha_Alta:** ahora la escribe el Flujo 1 en la misma actualización que la clasificación, con `formatDateTime(triggerOutputs()?['body/createdon'], 'yyyy-MM-dd')`. Antes quedaba vacía. Verificado con INC-0087.

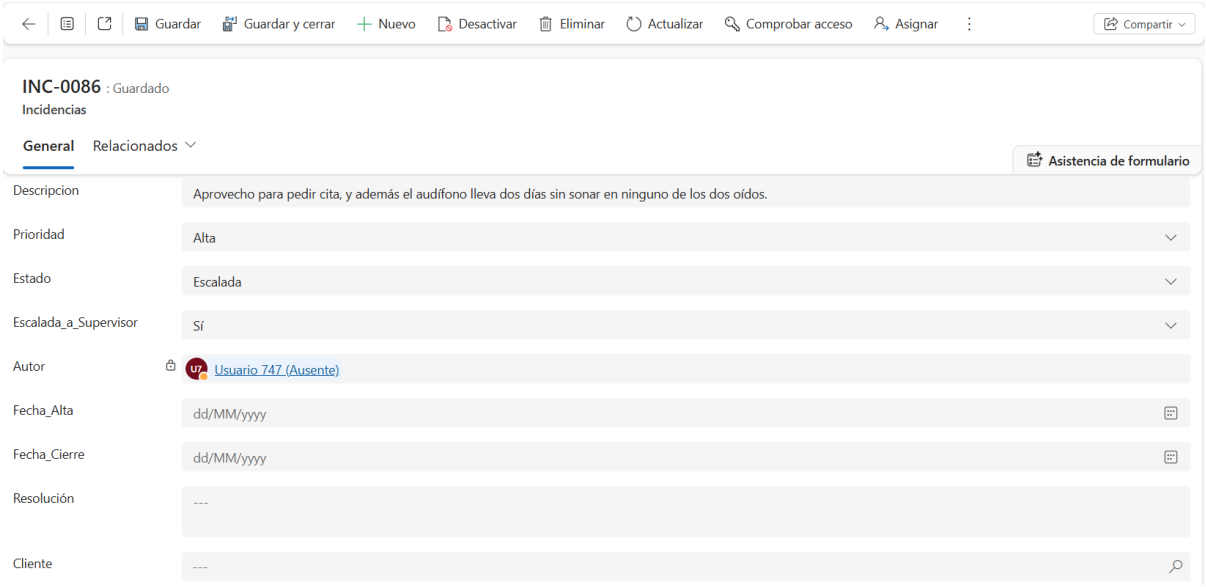


Figura 9. *Ficha de la incidencia con Autor (solo lectura), Fecha_Cierre y Resolución.*

9. Flujo 3 — Cierre con resolución obligatoria

9.1 Objetivo funcional

Que ninguna incidencia se cierre sin dejar escrito **cómo se resolvió**, que la fecha de cierre sea fiable y que quien la registró **se entere**, para poder informar al cliente. El aviso directo al cliente queda fuera a propósito: requiere otro canal y otras garantías de protección de datos.

9.2 Diseño

- **Desencadenador:** Dataverse, tipo de cambio *Actualizar*, tabla Incidencias, ámbito Organización, **Seleccionar columnas** `cr3f3_estado` y **Filtrar filas** `cr3f3_estado eq 610150003`. Solo arranca cuando el *Estado* cambia a Cerrada, no con cualquier edición.
- **Condición:** `empty(triggerOutputs()?['body/cr3f3_resolucion'])` es igual a `true`.
- **Rama sin resolución:** *Actualizar una fila* → Estado = **En curso**. La incidencia «rebota».
- **Rama con resolución:** *Actualizar una fila* → `Fecha_Cierre = utcNow()`; *Obtener una fila por id* en la tabla Usuarios con el `_createdby_value` del desencadenador (columnas `internalemailaddress,fullname`); Teams *Publicar mensaje* (Bot de Flow → chat con el autor): «✅ Incidencia «INC-xxxx» cerrada. Resolución: … Informa al cliente del resultado».

9.3 Por qué no se crea un bucle

El flujo modifica la misma fila que lo dispara, pero el desencadenador solo vigila `cr3f3_estado` y solo cuando vale *Cerrada*. Escribir *En curso* o *Fecha_Cierre* no cumple el filtro. **Filtrar en el desencadenador** (y no con una condición dentro del flujo) evita además ejecuciones inútiles.

9.4 Pruebas con INC-0086

Prueba	Resultado
Cerrada **sin** resolución (11:21)	Condición `true` → vuelve a *En curso*. Correcto en 601 ms
Cerrada **con** resolución (13:04)	Condición `false` → Fecha_Cierre 19/10/2026, autor localizado y aviso en Teams. Correcto en 3 s

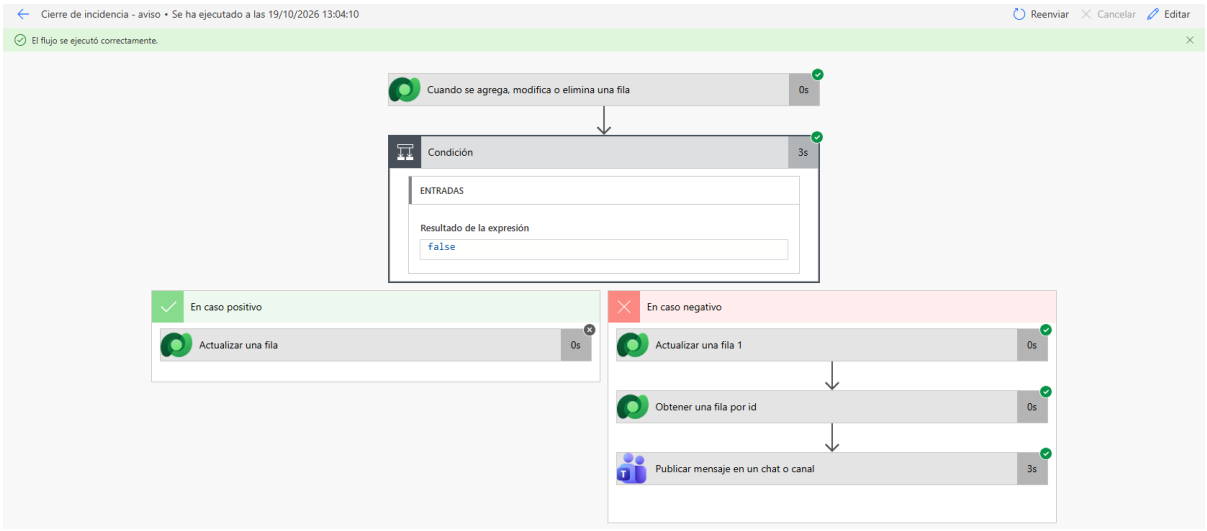


Figura 10. *Ejecución con resolución: fecha de cierre, lectura del autor y aviso por Teams.*

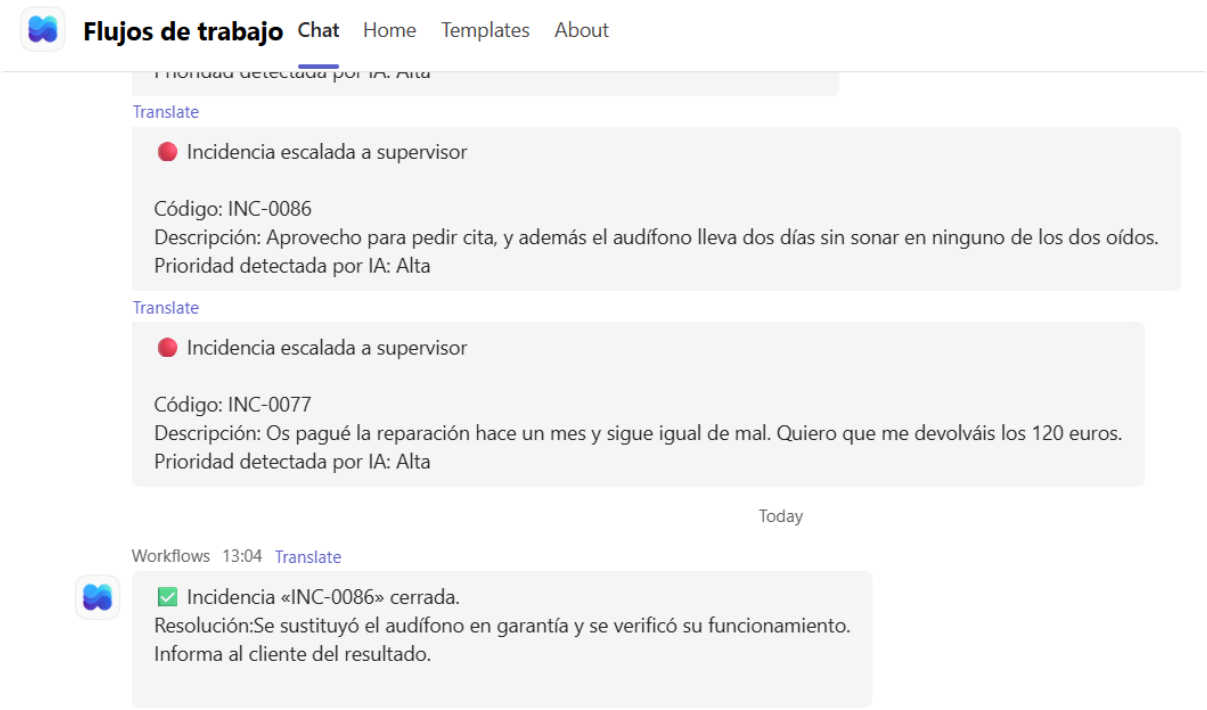


Figura 11. *Ciclo completo en Teams: aviso de escalado (Flujo 1) y aviso de cierre (Flujo 3).*

10. Flujo 4 — Asignación «Me encargo» con tarjeta adaptable

10.1 Objetivo funcional

Que una escalada no dependa de que el supervisor lea un mensaje y vaya a la aplicación. La tarjeta permite **hacerse cargo con un clic**, y el sistema registra **quién** se encargó y **cuándo** pasó a estar en curso.

10.2 Por qué un flujo separado

Publicar tarjeta adaptable y esperar una respuesta deja la ejecución **esperando** hasta que alguien pulsa, lo que pueden ser horas. Meterlo en el Flujo 1 bloquearía la clasificación. Separarlo además desacopla: el Flujo 4 reacciona a *Escalada* venga de donde venga (IA o cambio manual).

10.3 Diseño

1. **Desencadenador:** Dataverse, *Actualizar*, Seleccionar columnas `cr3f3_estado`, Filtrar filas `cr3f3_estado eq 610150002`.
2. **Teams — Publicar tarjeta adaptable y esperar una respuesta** en el canal *Soporte AudioVital / General*. Tarjeta Adaptive Card 1.4 con título, código, prioridad, descripción y un `Action.Submit` «🙋 Me encargo». Al responder, la tarjeta se sustituye por «✅ Asignada. Gestión en curso».
3. **Enumerar filas** en Usuarios con `domainname eq '<responder/userPrincipalName>'` y `1` fila: busca en Dataverse a quien pulsó.
4. **Actualizar una fila:** Estado `=` En curso y Propietario `=` `concat('systemusers(', first(outputs('Enumerar_filas')?['body/value'])?['systemuserid'], ')')`.

10.4 Error encontrado y corregido

La primera ejecución falló en *Actualizar una fila* con **BadRequest**: `The supplied reference link -- systemusers() -- is invalid`. *Enumerar filas* había terminado «correctamente»… pero **sin resultados**. El filtro buscaba por `internalemailaddress` con el correo de la respuesta, y Teams identifica a quien responde por su **UPN** (nombre de inicio de sesión). Se cambió a `domainname` (el campo que guarda el UPN en Dataverse) y `responder/userPrincipalName`. Tras **Reenviar**: todo en verde (tarjeta 12 s, búsqueda 0,1 s, actualización 0,5 s) e INC-0087 pasó de *Escalada* a *En curso* con su propietario.

> *Lección: en automatización, un paso «correcto» puede devolver cero filas. Antes de usar first() sobre una consulta conviene comprobar que no viene vacía y tener una ruta alternativa.*

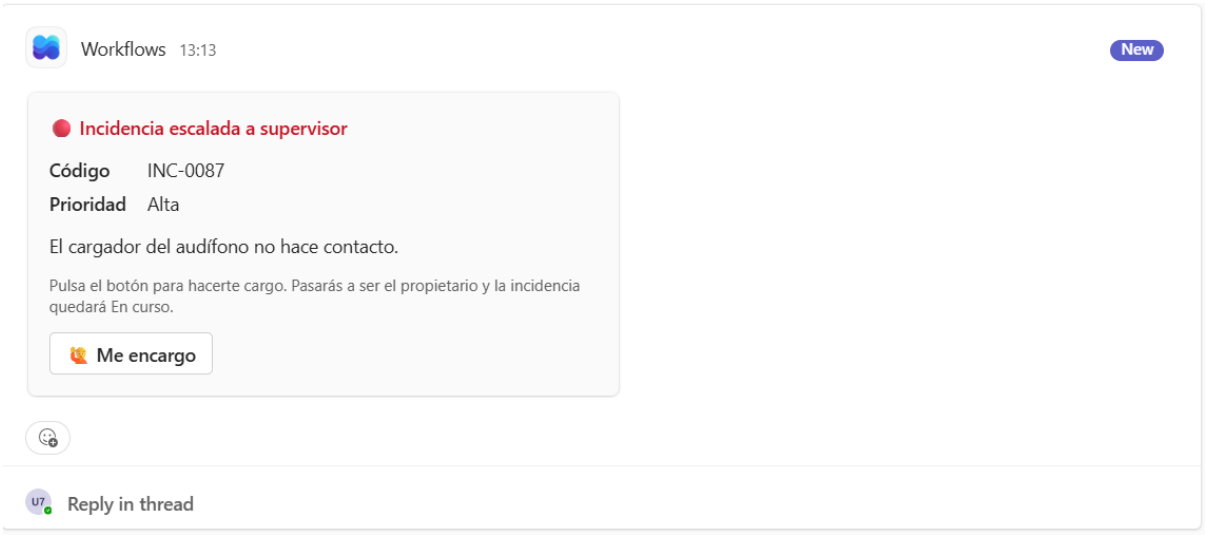


Figura 12. *Tarjeta adaptable publicada en el canal al escalar INC-0087.*

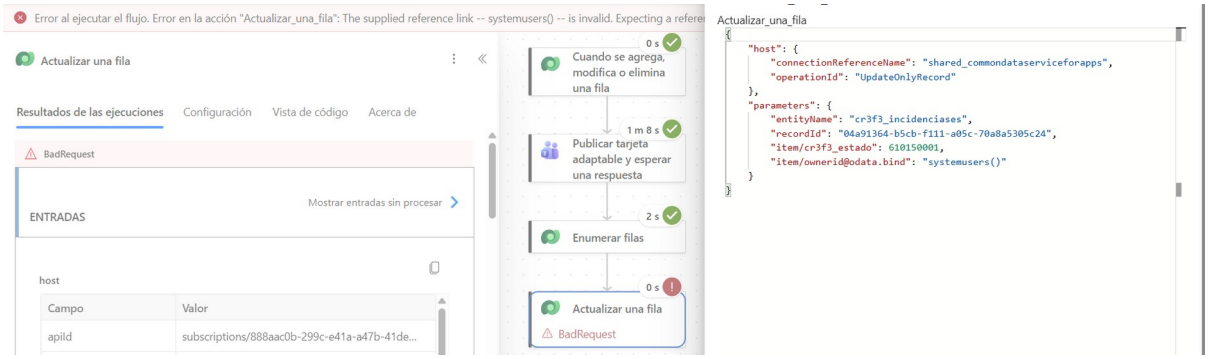


Figura 13. *Diagnóstico del BadRequest: el propietario llegó vacío porque la búsqueda no encontró al usuario.*



Figura 14. *INC-0087 en curso tras pulsar «Me encargo».*

11. Problemas resueltos en la Fase 3

Problema	Causa	Solución
Conexión a Key Vault rota tras crearla	Se pegó el *Id.* del secreto de cliente en lugar de su *Valor*	Reparar la conexión con el Valor (solo se muestra al crearlo)
El flujo parecía no haberse guardado	Guardado interrumpido	«Recuperar flujo» y volver a guardar
Respuestas de 5 min y 429	Bucle de autodisparo + prompt de 247 KB	Condición antibucle y Enumerar filas filtrado (sección 7)
Respuestas duplicadas o triplicadas	El mismo bucle	Ídem
El asistente atribuía incidencias a la clienta como si las hubiera «registrado»	El prompt no explica la diferencia Cliente / Creado por / Propietario	Detectado; ajuste del prompt pendiente (sección 12)
Valor `true` de una condición tratado como texto	Escrito a mano en lugar de como expresión	Introducirlo con fx
«Recipient» de Teams no admitía una expresión	Campo en modo selector de personas	⚙️ → «Usar contenido dinámico»
La primera ejecución del Flujo 3 tardó en aparecer	Registro inicial del desencadenador de Dataverse	Esperar o desactivar/activar el flujo
Fecha_Alta vacía en todas las incidencias	Ningún flujo la escribía	Flujo 1: `formatDateTime(createdon)` en la misma actualización
Dos columnas llamadas «Estado»	Estado propio y *statecode* del sistema (Activo/Inactivo)	Identificar la correcta por sus opciones
BadRequest `systemusers()`	Búsqueda por correo; Teams devuelve el UPN	Buscar por `domainname` con `userPrincipalName`
Correo personal visible en capturas	Cabeceras y conexiones del portal	Revisión con OCR y tachado antes de publicar

12. Lo que no se construyó

Parte del alcance previsto para la Fase 3 no se ejecutó antes de cerrar el proyecto (caducidad del crédito de Azure el ~21/10 y de la prueba de Teams el 23/10). Se declara para que no se dé por hecho:

Pieza prevista	Estado	Cómo se haría
Rama de error del Flujo 1 → *Pendiente de revisión*	Estado creado; la rama sigue solo avisando por Teams	Añadir *Actualizar una fila* (Estado 610150005) en la rama «Ejecutar después de: ha fallado»
Recordatorio de escaladas sin asignar fuera de plazo	No construido	Flujo programado cada hora: Escaladas con `modifiedon` > plazo del protocolo → aviso al responsable superior
Evitar el doble aviso de escalado	Flujo 1 (texto) y Flujo 4 (tarjeta) avisan los dos	Quitar el mensaje de texto de la rama de escalado del Flujo 1
Comprobar si la búsqueda del Flujo 4 viene vacía	No construido	Condición `empty()` antes de actualizar y aviso al canal
Roles de seguridad de Dataverse	No construido	Agente (crear y ver lo suyo), Supervisor (ver todo y asignar), Administrador
Tabla `Registro_Ejecuciones`	No construido; sustituido en parte por Log Analytics	Una fila por ejecución: incidencia, modelo, tokens, latencia, resultado, error
Auditoría de Dataverse	Desactivada	Activar en el entorno y en la tabla Incidencias
Bandeja del supervisor y Power BI ampliado	No construido	Vista «Pendientes» y página de tiempos de asignación y resolución
Semántica de columnas en el prompt del Flujo 2	Detectado, no corregido	Explicar Cliente / Creado por / Propietario en el mensaje de sistema

13. Límites conocidos (consolidados)

Del entorno

- Inquilino de formación con **un solo usuario**: la asignación y los avisos funcionan con identidades reales, pero no se ha podido demostrar con varios agentes y supervisores.
- **Infraestructura temporal:** crédito de Azure hasta ~21/10/2026 y prueba de Teams hasta 23/10/2026. Después, la solución queda documentada y demostrable con evidencias, pero no ejecutable. El entorno de Power Platform (tipo Desarrollador) no caduca.
- **Azure AI Search en plan Gratis:** sin reordenación semántica (cuota agotada en la Fase 2), sin réplicas ni SLA.
- El modelo `gpt-4.1-mini` está en estado **Legacy** y se retira el 14/4/2027.

De seguridad

- Entidad de servicio con **secreto de cliente**, no identidad administrada; rotación manual.
- Key Vault con **acceso público** (protegido por Entra ID y RBAC, sin punto de conexión privado).
- Las **salidas seguras** ocultan el prompt al depurar.
- Los flujos se ejecutan con conexiones de la cuenta del autor, no con una cuenta de servicio.

Funcionales

- El asistente solo ve las **50 incidencias más recientes**: los recuentos globales («¿cuántas…?») pueden ser incompletos.
- Doble aviso de escalado (texto y tarjeta) hasta retirar el del Flujo 1.
- Evaluación de la clasificación con 45 casos de un solo autor (ver memoria de las Fases 1 y 2).

14. Hoja de ruta para llevarlo a producción

Lo construido demuestra el patrón completo, pero un despliegue real en una organización exigiría lo siguiente. Se ordena por áreas y, al final, en fases.

14.1 Gestión del ciclo de vida (ALM)

- **Tres entornos** de Power Platform —Desarrollo, Pruebas y Producción— como **entornos administrados**, con la aplicación, las tablas y los cuatro flujos dentro de una **solución**.
- **Solución no administrada en Desarrollo, administrada en Pruebas y Producción**, desplegada con **Power Platform Pipelines** o con GitHub Actions / Azure DevOps (Power Platform Build Tools). La solución se exporta y se versiona en el repositorio.
- **Referencias de conexión y variables de entorno** para todo lo que cambia entre entornos: URL e índice de AI Search, nombre del Key Vault, despliegue de OpenAI, equipo y canal de Teams. Nada escrito a mano en los flujos.
- **Variables de entorno de tipo secreto vinculadas a Azure Key Vault**, en lugar de la acción *Obtener secreto* con una entidad de servicio propia.

14.2 Identidad, acceso y seguridad

- Flujos propiedad de una **cuenta de servicio** (o usuario de aplicación de Dataverse), con copropietarios del equipo de IT. Ninguna conexión a nombre de una persona.
- **Roles de seguridad de Dataverse** (Agente, Supervisor, Administrador) y **unidades de negocio** por tienda o zona, para que cada agente vea solo lo suyo y el supervisor lo de su zona.
- **Grupos de Entra ID** para asignar roles, MFA y acceso condicional.
- **Azure:** Key Vault y AI Search con **punto de conexión privado** y red virtual (compatibilidad de Power Platform con redes virtuales en entornos administrados); acceso a AI Search con **RBAC** (*Search Index Data Reader*) en lugar de claves; **identidades administradas** entre AI Search, Storage y Azure OpenAI para el indexador; rotación automatizada de secretos o uso de certificados.
- **Directivas de prevención de pérdida de datos (DLP)** que solo permitan los conectores aprobados (Dataverse, Teams, Azure OpenAI, Key Vault, HTTP restringido).

14.3 Datos personales y cumplimiento

- Las descripciones de incidencias pueden contener **datos de salud** (pérdida de audición): categoría especial del RGPD. Requiere **evaluación de impacto (EIPD)**, base legal, minimización y retención definida.
- Despliegues de Azure OpenAI en **zona de datos de la UE** y recursos en regiones europeas (hoy todo está en East US).
- **Minimizar lo que se envía al modelo:** no la tabla de incidencias, sino solo los campos imprescindibles; nunca datos de contacto del cliente.
- **Auditoría de Dataverse** activada y política de retención de los registros de diagnóstico.

14.4 IA generativa y RAG

- **Modelo vigente** (no Legacy) y despliegue con la cuota adecuada; valorar capacidad aprovisionada si el volumen lo justifica.

- **Azure AI Search en plan Básico o Estándar**, con **reordenación semántica** de pago y al menos **2 réplicas** (requisito del SLA de lectura; 3 para lectura y escritura).
- **Indexación programada** o por eventos cuando cambie un documento; estrategia de fragmentación con solapamiento; **metadatos** (versión, fecha de vigencia, departamento) y **filtros de seguridad** si hay documentos restringidos.
- **Citas con enlace** al documento y a su versión, para que el agente pueda comprobar la fuente.
- **Evaluación continua:** el conjunto de 45 casos como prueba de regresión antes de cambiar prompt o modelo, ampliado con incidencias reales anonimizadas y un segundo anotador. Para el RAG, medir la recuperación (si el documento correcto está entre los *k* primeros) y la fidelidad de la respuesta a la fuente.
- **Prompts versionados** en la solución, con pruebas antes de publicarlos.
- Sustituir el asistente por palabra clave por un **agente de Copilot Studio** publicado en Teams, con Dataverse y AI Search como herramientas: consultas filtradas en lugar de las 50 últimas filas.

14.5 Fiabilidad y operación

- Patrón **try/catch con ámbitos** en todos los flujos, rama de error que deja la incidencia en *Pendiente de revisión* y aviso al equipo de soporte.
- **Control de simultaneidad** y comprobaciones de vacío antes de cada `first()`; paginación y consultas filtradas en lugar de listas completas.
- **Alertas** sobre Log Analytics (errores 429, latencia, fallos de llamadas) y sobre ejecuciones fallidas de flujos; tabla `Registro_Ejecuciones` para métricas de negocio en Power BI.
- Revisar los **límites de peticiones de Power Platform** por licencia y el **coste** de Azure con presupuestos por entorno.

14.6 Licencias y coste

- Los conectores **HTTP, Azure OpenAI y Key Vault son premium**: Power Automate Premium por usuario, o licencia de proceso para los flujos de la cuenta de servicio; Power Apps por usuario o por aplicación.
- Teams para todos los agentes y supervisores; Power BI Pro (o capacidad) para compartir el panel.
- Azure: OpenAI por tokens (céntimos por incidencia con prompts acotados), AI Search de pago por réplica, Key Vault y Log Analytics de coste bajo.

14.7 Adopción

- **Piloto en 2–3 tiendas** durante unas semanas, comparando la prioridad del sistema con la del supervisor y midiendo el tiempo hasta asignación y hasta cierre.
- El supervisor puede **corregir** la clasificación; las correcciones alimentan la evaluación.
- Formación breve, guía de uso y canal para reportar errores; responsables claros (contenido, plataforma, calidad).

14.8 Orden propuesto

Etapa	Contenido	Criterio para pasar a la siguiente
1 · Bases	Entornos, solución, cuenta de servicio, variables de entorno, DLP, roles	Despliegue reproducible de Desarrollo a Pruebas sin tocar flujos a mano
2 · Seguridad y datos	EIPD, región UE, red privada, RBAC en AI Search, auditoría	Visto bueno de seguridad y protección de datos

Etapa	Contenido	Criterio para pasar a la siguiente
3 · Calidad	Modelo vigente, búsqueda semántica, evaluación ampliada, alertas, try/catch	Acierto y escalados perdidos dentro del umbral acordado
4 · Piloto	2–3 tiendas con supervisión humana de la clasificación	KPIs del piloto y satisfacción de agentes y supervisores
5 · Despliegue	Resto de tiendas por oleadas, formación y soporte	Operación estable y revisiones periódicas de calidad

15. Evidencias

Las 45 capturas de la Fase 3 están en `capturas/fase3/` del repositorio, con su índice. Todas se revisaron antes de publicar: ningún secreto visible y los correos de la cuenta tachados.

Capturas	Tema
17 – 26	Entra ID, Key Vault, RBAC, claves de consulta, conexión con entidad de servicio y prueba en Teams
27 – 29	Respuesta de 5 minutos, salidas seguras e historial de reintentos con 429
30 – 38	Límite del despliegue, bucle de autodisparo, condición antibucle, Enumerar filas filtrado y respuestas corregidas
39 – 43	Costes por recurso, métricas de tokens, presupuesto, diagnóstico y consulta KQL en Log Analytics
44 – 45	Formulario con Autor y Resolución
46 – 53	Flujo 3: configuración, pruebas sin y con resolución y avisos en Teams
54 – 55	Fecha_Alta automática en el Flujo 1
56 – 61	Flujo 4: tarjeta «Me encargo», error de la búsqueda por correo, corrección y resultado

> *Proyecto personal de portafolio desarrollado por José Rafael Vázquez Silva con asistencia de una herramienta de IA para el diseño guiado y la documentación. Todas las configuraciones se hicieron, probaron y verificaron en el entorno real.*